

UNIVERSIDAD AMERICANA
Facultad de Ingeniería y Arquitectura
Ingeniería en Sistemas de la Información



Algebra Lineal

Grupo 8:

- Alex Ivan Díaz Yañez
- María Regina Medrano Flores
- Jesser Jadiel Rodriguez Chavarria
- Carlos Fernando Torres Gonzalez

Docente:

José Andrés Munguía Cortez

Fecha: 08/09/2026

Reducción hacia adelante (forma escalonada)

El algoritmo recorre cada columna j , de izquierda a derecha, buscando construir un pivote en la fila actual (fila_pivote). Este avance es lo que primero lleva la matriz a una forma escalonada; los ceros por encima del pivote se agregan después, en la Fase 2.

Selección inteligente del pivote (evitar fracciones)

A diferencia de una eliminación gaussiana básica —que simplemente toma la fila con el mayor valor absoluto—, este módulo intenta primero encontrar una fila que ya tenga un 1 o un -1 en la columna j :

```
fila_elegida = -1
for i in range(fila_pivote, m):
    if abs(Ab[i][j]) == 1:
        fila_elegida = i
        break

# Si ninguna fila tiene 1 o -1, se usa pivoteo parcial
# (la fila con mayor valor absoluto en la columna j)
```

Esta estrategia es una optimización práctica: si el pivote ya es 1 o -1 , la normalización posterior no genera fracciones nuevas, y las operaciones que se muestran en pantalla resultan más simples de leer. Solo cuando ninguna fila cumple esa condición se recurre al pivoteo parcial clásico (mayor valor absoluto), que sigue garantizando estabilidad numérica.

Intercambio de filas

Si la fila elegida no es la fila actual, se intercambian y se registra el paso:

```
pasos.append(("pivoteo",
    f"Selección de Pivote Óptimo: Fila {fila_pivote+1} ↔ Fila {fila_elegida+1} "
    f"(para evitar fracciones)", [f[:] for f in Ab]))
```

Pivote igual a 1

El requisito del enunciado —"pivotes iguales a 1"— se cumple en dos pasos posibles:

- Si el pivote es exactamente -1 , primero se multiplica toda la fila por -1 .
- Si el pivote es distinto de 1 (y de -1 ya corregido), se divide toda la fila entre el valor del pivote.

```
if Ab[fila_pivote][j] == -1:
    for k in range(j, n + 1):
        Ab[fila_pivote][k] *= -1

pivote_val = Ab[fila_pivote][j]
if pivote_val != 1:
    for k in range(j, n + 1):
        Ab[fila_pivote][k] /= pivote_val
```

Cada una de estas transformaciones también se agrega a la lista pasos, para poder mostrarla después en pantalla.

Ceros debajo del pivote

Una vez que el pivote de la columna j vale 1, se recorren todas las filas debajo de fila_pivote y se les resta un múltiplo de la fila pivote, de forma que su elemento en la columna j quede en cero:

```
for i in range(fila_pivote + 1, m):
    factor = Ab[i][j]
    if factor != 0:
        for k in range(j, n + 1):
            Ab[i][k] -= factor * Ab[fila_pivote][k]
    pasos.append(("operacion",
        f"F {i+1} = F {i+1} - ({factor}) * F {fila_pivote+1}", [f[:j] for f in Ab]))
```

Al terminar esta fase, la matriz ya está en forma escalonada: cada pivote vale 1 y todos los

elementos debajo de él son cero.

Detección automática de las columnas pivote

Cada vez que se establece un pivote válido, el programa guarda la posición (fila, columna) en la lista `pivotes_pos`:

```
pivotes_pos.append((fila_pivote, j))
```

Esta lista es la fuente de verdad para todo lo que viene después: de ella se derivan las columnas pivote que se listan en pantalla, la separación entre variables básicas y libres, y la forma en que se construye la solución final. No se necesita ningún paso adicional de "búsqueda" de pivotes: quedan registrados automáticamente en el mismo recorrido en que se calculan.

Si una columna no tiene ningún valor distinto de cero disponible (todo ceros desde `fila_pivote` hacia abajo), simplemente no se agrega ningún pivote para ella y el algoritmo pasa a la siguiente columna sin avanzar `fila_pivote`; esa columna queda asociada más adelante a una variable libre.

Verificación de inconsistencia

Antes de continuar hacia la Fase 2, el programa revisa si alguna fila quedó de la forma $[0 \ 0 \ \dots \ 0 \ | \ b]$ con b distinto de cero, lo cual indica una ecuación imposible de satisfacer (por ejemplo, $0 = 5$):

```
for i in range(m):
    if all(Ab[i][j] == 0 for j in range(n)) and Ab[i][n] != 0:
        return Ab, pivotes_pos, pasos, True # sistema inconsistente
```

El cuarto valor que retorna la función (True/False) indica precisamente si el sistema es inconsistente, y la interfaz lo usa para detener el proceso y mostrar que no existe solución, sin necesidad de calcular una solución paramétrica que no tendría sentido.

Reducción hacia atrás (ceros por encima del pivote)

Aquí es donde la forma escalonada se convierte propiamente en RREF. El programa recorre los pivotes ya encontrados, en orden inverso (de derecha a izquierda / de abajo hacia arriba), y para cada uno elimina los elementos que quedan por encima de él en su columna:

Al finalizar esta fase, cada columna pivote tiene un 1 en su fila pivote y cero en todas las demás

filas (arriba y abajo). Esa es, por definición, la Forma Escalonada Reducida por Filas (RREF).

Columnas pivote

La lista de columnas pivote se obtiene extrayendo el segundo elemento de cada tupla en `pivotes_pos`:

```
cols_pivote = set(p[1] for p in pivotes_pos)
```

A partir de este conjunto, la interfaz arma el mensaje descriptivo, por ejemplo: "Las columnas pivote son: 1, 2 y 4", usando las posiciones (+1 porque el índice interno empieza en 0).

Variables básicas y variables libres

Con las columnas pivote identificadas, `construir_solucion_general()` separa las variables en dos grupos:

```
vars_basicas = [j for j in range(n) if j in cols_pivote]
vars_libres = [j for j in range(n) if j not in cols_pivote]
```

- Variables básicas: corresponden a una columna pivote; su valor queda determinado por la fila de ese pivote.
- Variables libres: no tienen columna pivote asociada; pueden tomar cualquier valor real y se muestran como parámetro ($\in \mathbb{R}$).

Construcción de la solución final

La función `construir_solucion_general()` decide qué tipo de solución mostrar según la cantidad de variables libres:

Solución única (0 variables libres)

Si no quedaron variables libres, el sistema es consistente y determinado. Como la matriz ya está en RREF, el valor de cada variable básica se lee directamente de la última columna (el término independiente) de su fila pivote, sin necesidad de ninguna sustitución hacia atrás:

```
lineas_solucion.append("Solución Única (Sistema Consistente Determinado):")
```

```
for f, c in pivotes_pos:
    lineas_solucion.append(f" x{c+1} = {a_fraccion_str(Ab[f][n])}")
```

Solución general paramétrica (≥ 1 variable libre)

Si existen variables libres, el sistema tiene infinitas soluciones. Para cada variable básica, su ecuación en la fila pivote se despeja en función de las variables libres, tomando el término independiente y restando el efecto de cada columna libre presente en esa fila:

```
for f, c in pivotes_pos:
    terminos = [termino_independiente]
    for jl in vars_libres:
        coef = Ab[f][jl]
        if coef != 0:
            # se agrega el término -coef * x_jl con su signo correspondiente
            ...
    lineas_solucion.append(f" x{c+1} = {expresion}")

for jl in vars_libres:
    lineas_solucion.append(f" x{jl+1} (libre,  $\in \mathbb{R}$ )")
```

El resultado es una lista de líneas de texto lista para mostrarse en pantalla, con una ecuación por cada variable básica en términos de las variables libres, seguida de la declaración explícita de cuáles variables quedan libres.

Casos de prueba

Caso 1: Sistema con solución única

1. Eliminación Principal ($Ax=b$)

2. Reducción RREF (Gauss-Jordan)

3. Ecuación Matricial (Ax)

Forma Escalonada Reducida, Detección de Pivotes y Variables Básicas/Libres

Filas (m): Columnas (n): Generar Matriz

x_1	x_2	x_3	=	b
1	0	0	=	2
0	1	0	=	-1
0	0	1	=	3

Reducir a RREF

=== PROCESAMIENTO: GAUSS-JORDAN (SELECCIÓN DE PIVOTE ÓPTIMO) ===

1. FORMA ESCALONADA REDUCIDA FINAL (RREF):

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 3 \end{array} \right]$$

2. DETECCIÓN DE PIVOTES:
Las columnas pivote son: 1, 2, 3

3. CLASIFICACIÓN DE VARIABLES:

- Variables Básicas: x_1, x_2, x_3
- Variables Libres: Ninguna

4. ESTRUCTURA DE LA SOLUCIÓN FINAL:

$x_1 = 2, x_2 = -1, x_3 = 3$

Solución Unica (Sistema Consistente Determinado):

$x_1 = 2$
 $x_2 = -1$
 $x_3 = 3$

Caso 2: Sistema con infinitas soluciones

1. Eliminación Principal ($Ax=b$)

2. Reducción RREF (Gauss-Jordan)

3. Ecuación Matricial (Ax)

Forma Escalonada Reducida, Detección de Pivotes y Variables Básicas/Libres

Filas (m): Columnas (n): Generar Matriz

x_1	x_2	x_3	=	b
1	1	1	=	3
2	2	2	=	6
3	3	3	=	9

Reducir a RREF

=== PROCESAMIENTO: GAUSS-JORDAN (SELECCIÓN DE PIVOTE ÓPTIMO) ===

[↴] Operación: $F_2 = F_2 - (2) * F_1$

[	1	1	1		3	]
[	0	0	0		0	]
[	3	3	3		9	]

[↴] Operación: $F_3 = F_3 - (3) * F_1$

[	1	1	1		3	]
[	0	0	0		0	]
[	0	0	0		0	]

1. FORMA ESCALONADA REDUCIDA FINAL (RREF):

[	[	1	]	1	1		3	]
[	0	0	0		0	]		
[	0	0	0		0	]		

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2. DETECCIÓN DE PIVOTES:

Las columnas pivote son: 1

3. CLASIFICACIÓN DE VARIABLES:

- Variables Básicas: x_1
- Variables Libres: x_2, x_3

4. ESTRUCTURA DE LA SOLUCIÓN FINAL:

Solución General Paramétrica (2 variable(s) libre(s)):

$$x_1 = 3 - x_2 - x_3$$

$$x_2 \text{ (libre, } \in \mathbb{R})$$

$$x_3 \text{ (libre, } \in \mathbb{R})$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

DIAGNÓSTICO DEL SISTEMA MATEMÁTICO:

-> ESTADO: Sistema Consistente Indeterminado (2 variables libres).

Caso 3: Sistemas sin solución

Forma Escalonada Reducida, Detección de Pivotes y Variables Básicas/Libres

Filas (m):

3

Columnas (n):

3

Generar Matriz

x_1	x_2	x_3	=	b
1	2	3	=	4
2	4	6	=	8
1	2	3	=	7

Reducir a RREF

=== PROCESAMIENTO: GAUSS-JORDAN (SELECCIÓN DE PIVOTE ÓPTIMO) ===

[↴] Operación: $F_2 = F_2 - (2) * F_1$

[	1	2	3		4	]
[	0	0	0		0	]
[	1	2	3		7	]

[↴] Operación: $F_3 = F_3 - (1) * F_1$

[	1	2	3		4	]
[	0	0	0		0	]
[	0	0	0		3	]

1. FORMA ESCALONADA REDUCIDA FINAL (RREF):

[	[	1	]	2	3		4	]
[		0		0	0		0	]
[		0		0	0		3	]

[4] Operación: $F_3 = F_3 - (1) * F_1$

[	1	2	3		4	]
[	0	0	0		0	]
[	0	0	0		3	]

1. FORMA ESCALONADA REDUCIDA FINAL (RREF):

[	[	1	]	2	3		4	]
[	0	0	0		0	]		
[	0	0	0		3	]		

DIAGNÓSTICO DEL SISTEMA:

-> ESTADO: Sistema Inconsistente (Sin Solución). Existe una fila $[0 \dots 0 \mid c]$ con $c \neq 0$.